The Protocol Gap Is Not a Property of Your Model: Leakage and Information in YelpCHI Fake Review Detection

Rohan Mukka
School of Computer Science
University of Oklahoma
Norman, OK, USA
rohan.mukka-1@ou.edu

Rithwik Reddy Donthi Reddy
School of Computer Science
University of Oklahoma
Norman, OK, USA
rithwikreddyd@ou.edu

***Abstract*—Fake reviews in YelpCHI are concentrated by business: 657 of its 1,224 businesses contain none at all, and the per-business fake rate alone is a 0.951-AUC predictor of the label. The usual remedy is to group cross-validation folds so no business spans the split, which produces a dramatic collapse — a behavioral random forest falls from 0.934 to 0.646 AUC-ROC. We show that reading is backwards. Business-level quantities are of three kinds the protocol conflates: statistics derived from labels, which are leakage; the identity of a known target, which the split correctly removes; and label-free profiles of a business's own reviews, which a platform observes for a new listing and which transfer to unseen businesses at 0.805 AUC. A strict training-only featurization removes all three. Restoring only the third, using no labels, returns 0.252 of that 0.289. Measured inside that regime the split costs behavioral models 0.021 AUC on average against 0.056 for text models it cannot affect: given honest features, behavioral models transfer to unseen businesses *better* than textual ones on average, while scoring far higher. The protocol gap is a property not of a model but of the pair (model, feature regime), and measuring it in one regime can invert an ordering rather than merely rescale it. Graph models compound this — a node's degree is worth 0.709 AUC under one split and 0.708 under the other — so grouping is necessary but not sufficient for them.**

***Index Terms*—fake review detection, evaluation methodology, data leakage, YelpCHI, reviewer behavior, focal loss, multi-modal fusion**

I. Introduction

Online review platforms are consequential enough that manipulating them is profitable. Luca and Zervas estimate that review fraud is economically significant at the scale of entire markets [1], and the availability of fluent text generation has lowered the cost of producing plausible fake content [2]. A large research literature has responded with detectors that combine review text with reviewer behavior, and reported numbers on the standard benchmark — YelpCHI [3] — are strong.

This paper began as an attempt to build such a detector, and turned into an argument about how one should be measured.

The starting observation is that YelpCHI was assembled by collecting reviews for a set of Chicago businesses, and spam campaigns target particular businesses. The label is therefore concentrated by business rather than spread uniformly: 657 of the 1,224 businesses in the corpus contain zero fake reviews, the median business has a fake rate of exactly zero, and mapping each review to the fake rate of its business yields an AUC of 0.951 on its own. That figure makes every business-level quantity look like a shortcut, and suggests an obvious remedy — group the cross-validation folds so that no business spans the split, and a model can no longer profit from recognizing a business it has seen targeted before.

Doing so produces a dramatic collapse. A random forest over behavioral features falls from 0.934 to 0.646 AUC-ROC, enough to reverse the apparent ranking of behavioral and textual features. Read on its own, that looks like a demonstration that behavioral performance on this benchmark was mostly memorization.

That reading is backwards, and the experiments below say so. Business-level quantities are not one thing, and the split does not treat them alike. Statistics derived from labels are leakage and must go. The identity of a known target does not transfer, and grouping removes it correctly. But a label-free profile of a business's own reviews — how long they run, how tightly its ratings cluster — is something a platform observes for a listing created yesterday, and it transfers to businesses never seen during training at 0.805 AUC. A featurization strict enough to remove the first two typically removes the third as well, and the resulting number understates the method rather than correcting it. Ours did, which is how we came to notice — and once the third is restored, the behavioral models turn out to transfer to unseen businesses better than the textual ones, not worse.

Graph models make the same point from the other direction, and more sharply. A node's degree in the same-business relation is worth 0.709 AUC under the reviewer-disjoint split and 0.708 under the business-disjoint one: the split moves it by 0.001, because a degree is read off the whole graph rather than computed from rows a split can partition. For a transductive graph model, grouping folds by business does not remove business information at all; it merely moves it from the feature matrix into the topology.

We therefore treat the evaluation as having two axes rather than one — how folds are split, and where a feature's supporting statistics may come from — and report results across both.

Our contributions are:

1) A separation of the three business-level quantities that the conventional protocol conflates — label-derived statistics, business identity, and label-free business profiles — with a measurement of what each is worth (Section III).
2) A two-axis evaluation, varying the split and the source of a feature's statistics independently, which shows that the apparent collapse under a business-disjoint split is part leakage removal and part information removal, and separates them (Section VI-E).
3) Evidence that a business-disjoint split is necessary but not sufficient for a transductive graph model, because message passing reconstructs from the topology what the split withheld from the features (Section VI-G).
4) ReviewGuard, a two-branch detector fusing lexical text features with reviewer and business behavioral features, with per-model threshold tuning so that baselines are not handicapped — and a negative result about it: its fusion benefit does not survive the second axis (Section IV).
5) A data integrity audit of a widely circulated CSV distribution of YelpCHI, identifying label-contaminated timestamps and an uninformative behavioral feature file, both excluded from our experiments (Section III).

II. Related Work

A. Detection Methods

Early work treated deceptive review detection as text classification. Ott et al. [4] showed that n-gram and psycholinguistic features separate solicited deceptive reviews from genuine ones with near-human accuracy in a controlled corpus. Jindal and Liu [5] introduced opinion spam as a distinct problem and applied duplicate detection and engineered features at scale. Mukherjee et al. [6] made the central observation that motivates two-branch designs: reviewer behavior — posting bursts, rating extremity, activity patterns — carries signal that text alone does not, and combining the two beats either.

Representation learning followed. Recurrent and convolutional models replaced hand-built lexical features, and pretrained transformers [7], [8] became the default text encoder, consistently improving on bag-of-words baselines for review classification. Most recently the review ecosystem has been modeled as a heterogeneous graph over reviewers, businesses, and reviews. Rayana and Akoglu [3] propagate spam evidence through reviewer–product networks, and graph neural networks [9], [10] learn representations over the same structure. The YelpCHI distribution used by much of this line of work descends from Rayana and Akoglu's release, most often through the packaging released with CARE-GNN [10].

B. Evaluation and Leakage

Our concern is orthogonal to the choice of model. Kaufman et al. [11] characterize leakage as the introduction of information about the target that would be unavailable at prediction time, and document how easily it survives into published results. Benchmark-specific shortcut learning is now well documented across machine learning [12]: models reach high accuracy through features that correlate with the label in the dataset's construction rather than in the phenomenon.

Graph-based fake review detectors are the case where this matters most, because the reviewer–business graph makes business identity structural rather than incidental. This does not invalidate that work — a platform monitoring a fixed, known set of businesses may legitimately exploit business-level priors, and much of the deployment literature is in that setting. Two things are usually left unreported, though. The standard random or reviewer-grouped split does not distinguish a detector that generalizes to new businesses from one that has learned a lookup table of past targets; and, as Section VI-G shows, the natural correction does not fix this for a graph model, because its edges reconstruct what the split withheld from its features. We are not aware of published YelpCHI results reported in a way that separates these.

III. Data Integrity Audit

We work from a CSV distribution of YelpCHI containing 45,954 reviews with review text, reviewer and business identifiers, star ratings, timestamps, and labels; 6,677 reviews (14.53%) are labeled fake. The review counts, label counts, and class balance match the canonical `.mat` release. Before using any column we audited it. Two sources failed, and are excluded from every experiment in this paper. Table I summarizes the diagnostics.

TABLE I
Data integrity audit. The first two blocks describe inputs excluded from the study; the third describes a property of YelpCHI itself that dictates the evaluation protocol.

Source	Diagnostic	Value
`date` column	AUC from timestamp alone	0.965
	Monthly fake-rate range	0.026–0.498
	Distinct seconds values	1
`behavioral_`	Max $\|r\|$ vs. own definition	0.005
`features.csv`	Max $\|r\|$ vs. label	0.008
Business identity	Businesses with no fakes	657/1224
	AUC from business fake rate	0.951

A. Contaminated Timestamps

The released `date` column does not behave like a record of when reviews were posted. A random forest given nothing but the raw Unix timestamp separates fake from genuine reviews with an AUC of 0.965. The monthly fake rate swings from 49.8% in January 2010 to 2.6% in March 2010, and the entire corpus spans 111 days, against the multi-year range of the original Yelp collection. Meanwhile the within-day structure is absent: posting hours are uniform across all 24 hours to

within a factor of 1.11, and the seconds field takes exactly one distinct value.

The combination is diagnostic. Real posting times are strongly diurnal, so a uniform hour distribution and a constant seconds field indicate that the time component was generated rather than observed; and a label association this strong within a four-month window indicates that whatever generated the dates was conditioned on the class. Every temporal behavioral feature in the standard repertoire — posting burstiness, inter-review gaps, account age at review — would be computed from this column and would inherit the contamination. We therefore exclude timestamps entirely. This costs us the burstiness features that Mukherjee et al. [6] found informative, and we note it as a limitation rather than a design choice.

B. An Uninformative Feature File

The same distribution ships a six-column feature file, `behavioral_features.csv`, holding average star rating, review count, burst ratio, rating deviation, category diversity, and account age. Reconstructing each column from the review table it purports to summarize — for instance, computing each reviewer's actual mean rating — yields correlations of at most 0.005 with the shipped values. The columns also correlate with the label at most 0.008 in absolute value, and are near-degenerate: the burst ratio is zero for 97.5% of rows and category diversity is 1.0 for 99.4%. The file carries no recoverable information and we do not use it. We flag it because it is distributed alongside otherwise usable data and its filename invites use without inspection.

C. Business-Concentrated Labels

The third finding is a property of YelpCHI itself rather than a defect in a redistribution, and it is the one that shapes our evaluation. Fake reviews are concentrated by business: 657 of 1,224 businesses contain no fake reviews, 58 consist entirely of fake reviews, and the median business fake rate is zero. Mapping each review to its business's fake rate gives an AUC of 0.951. Fig. 1 shows the distribution.

This is not an error — it reflects how spam campaigns actually work, and how the corpus was collected. But it means business identity is close to a sufficient statistic for the label, and that makes any business-level quantity suspect. The temptation is to treat all of them as shortcuts and remove them. That is too blunt, and getting the distinction right turns out to matter more than the split itself. Three different things are easily conflated:

1) **Label-derived business statistics.** The observed fake rate of a business, or anything computed from it. This is the 0.951-AUC quantity above. It is unambiguous leakage — it cannot be known at prediction time — and it is never a feature in this paper.
2) **Business identity.** Knowing that *this particular* business has been targeted before. Available whenever a business appears on both sides of a split, and useless for a business never seen. This is what a business-disjoint split removes.
3) **Label-free business profiles.** How long this business's reviews run, how tightly its ratings cluster, how many distinct reviewers it has. These are computed from the business's own reviews without reference to any label, and a platform observes them for a brand-new listing.

The third category is not a shortcut, and treating it as one is a mistake we made before correcting it. Trained on one set of businesses and evaluated on a *disjoint* set, a profile of four such statistics reaches 0.906 review-level AUC and 0.805 AUC when scored once per business. That is not recognizing known spam targets; it is learning what a targeted business looks like and applying it to businesses never seen. The signal transfers, and a detector that discards it is handicapping itself for no methodological gain.

We therefore treat the source of a feature as a second experimental axis alongside the split (Section V-A), and report both. The distinction between the two lower categories is invisible under the conventional protocol, which is precisely the problem.

IV. ReviewGuard

The detector is a vehicle for the evaluation questions above rather than a contribution in itself, so we describe it compactly; the released code carries the full configuration.

Given a review $r = (t, b)$ with text t and behavioral context b, we learn $f : (t, b) \to [0, 1]$ estimating $P(\text{fake} \mid t, b)$.

Text branch (266 dims). Word unigram–bigram and character 3–5-gram TF-IDF, each capped at 50,000 features with minimum document frequency five and sublinear scaling, concatenated and reduced by truncated SVD to 256 dimensions. Character n-grams are included for robustness to the spelling and punctuation irregularity that distinguishes review registers. To this we append ten stylometric features in the tradition of Ott et al. [4]: character and word counts, mean word length, capitalization, exclamation, question, digit and punctuation ratios, first-person pronoun ratio, and type–token ratio.

Behavior branch (19 dims). Review-level: star rating, rating extremity $|\text{rating} - 3|$, word count. Reviewer-level: review count, singleton indicator, mean and standard deviation of ratings given, ratios of extreme and of positive ratings, distinct businesses reviewed, maximum reviews for a single business, mean review length. Business-level: review count, mean and standard deviation of ratings received, and the signed and absolute deviation of this review's rating from its business's mean. Two indicators, `rev_seen_in_training` and `prod_seen_in_training`, make the cold-start case explicit rather than hiding it in an imputed value. Where these aggregates are computed from is the second axis of Section V-A, and matters more than any other choice here.

Fusion. The two vectors are standardized, concatenated, and classified by a two-layer MLP (256 and 64 units, ReLU, dropout 0.3), trained with focal loss [13],

$$L = -\alpha_t (1 - p_t)^{\gamma} \log(p_t), \tag{1}$$

$\gamma = 2$ and α the training-set genuine-class frequency, against the 14.53% positive rate. We optimize with Adam [14] at

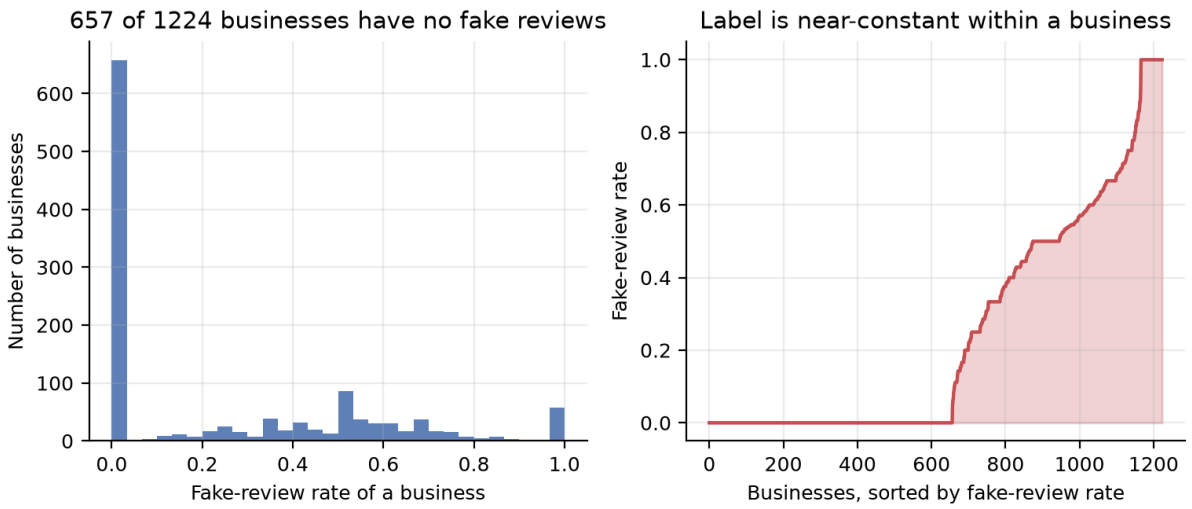

Fig. 1. Fake reviews are concentrated by business. Left: distribution of per-business fake rates; the mode at zero is 657 businesses with no fake reviews at all. Right: businesses sorted by fake rate, showing that the label is near-constant within a business.

10^{-3}, batch size 256, at most 40 epochs, keeping the best validation-AUC weights and stopping after six epochs without improvement. Single-branch ablations use the same architecture on one branch, with $(64, 32)$ units for the behavior-only model to match its input dimensionality.

A. On the Absence of a Transformer Branch

The design we set out to evaluate used a fine-tuned RoBERTa-base encoder [8] in place of the lexical representation. We report the lexical encoder instead for an infrastructural reason we state plainly: the compute environment for this study had no GPU and no network route to a pre-trained model repository. The transformer encoder is implemented and released with our code so the comparison can be completed where those resources exist.

This bounds our text-side numbers from below. It does not affect the paper's central claims, which concern the evaluation protocol and hold for any text encoder — though it does leave open whether a stronger text branch would restore the fusion benefit that Section VI finds reverses.

V. Experimental Setup

A. Two Axes

Evaluations here vary along two independent axes. The first is the split; the second is where a feature's supporting statistics may come from. Both are five-fold, stratified by label.

1) Split: **Business-disjoint (primary).** Folds are grouped by business identifier, so every business in a test fold is unseen during training. This is the setting in which a detector must generalize to businesses it has no history for.

Reviewer-disjoint (comparison). Folds are grouped by reviewer identifier, so no reviewer spans the split but businesses may. This corresponds to the precaution commonly taken in this literature.

2) Feature regime: **Inductive.** Reviewer and business aggregates are computed from the fold's training rows only. A business absent from training falls back to a training-set prior — a single constant, worth an AUC of 0.500 on its own. This is the strictest available reading.

Transductive (label-free). The same aggregates are computed over every review in the corpus, with labels untouched, so a held-out business is described by its own reviews. This is what a deployed detector observes for a new listing, and, as Section VI-G shows, what message passing over the review graph implicitly hands a graph model.

Neither regime uses a label at any point. We report both because the difference between them is the difference between removing leakage and removing information, and the two are easy to confuse: the inductive regime is safe by construction but discards the transferable business profiles of Section III, and so understates what behavioral features can do.

Within each fold we reserve a grouped, stratified validation slice (20% of the training portion, disjoint on the same variable) for early stopping and threshold selection. TF-IDF vocabularies, the SVD basis, and feature scalers are always fit on the fold's training portion alone, in both regimes.

B. Threshold Tuning

Under 14.53% positive prevalence, a classifier left at a 0.5 threshold predicts the majority class almost everywhere, and reporting F1 or recall at that threshold says more about calibration than about discrimination. We therefore tune a decision threshold for *every* model, including all baselines, on the validation slice to maximize Macro-F1, and apply it unchanged to the test fold. This matters for fair comparison: in a preliminary version of these experiments, untuned classical baselines recorded fake-class recalls as low as 0.003, which reflects an unadjusted threshold rather than a failure to rank.

C. Baselines

We compare against four baselines and two ablations: TF-IDF with a linear SVM [15] and with logistic regression, both on the sparse lexical matrix; behavioral features with logistic regression and with a random forest [17]; and the text-only and behavior-only MLPs, which serve as the fusion ablations.

D. Metrics and Significance

We report AUC-ROC, average precision, Macro-F1, and fake-class recall, as mean and standard deviation over the five folds. Average precision is included because it is the more informative summary under class imbalance. Pairwise comparisons between ReviewGuard and each baseline use the Wilcoxon signed-rank test over the five paired fold results. We note the limitation this imposes explicitly: with $n = 5$ paired observations, the smallest attainable two-sided p-value is 0.0625, so no comparison can reach $\alpha = 0.05$. We report the p-values for completeness and draw conclusions from effect sizes and fold-level consistency rather than from significance thresholds.

VI. Results

A. The Protocol Gap

Table II reports AUC-ROC for every model under both splits in the inductive feature regime; the models, features, hyperparameters, and tuning procedure are identical, and only the fold partitioning differs. Read on its own this table overstates its case, for reasons Section VI-E takes up; we present it first because it is the comparison the literature implicitly makes, and because the ordering within it is informative regardless.

TABLE II
EFFECT OF THE EVALUATION PROTOCOL ON AUC-ROC. THE REVIEWER-DISJOINT SPLIT ALLOWS A BUSINESS TO APPEAR IN BOTH TRAINING AND TEST FOLDS; THE BUSINESS-DISJOINT SPLIT DOES NOT. THE GAP IS THE APPARENT PERFORMANCE THAT DOES NOT SURVIVE THE STRICTER SPLIT. SECTION VI-E DECOMPOSES IT: FOR BEHAVIORAL MODELS MOST OF IT IS INFORMATION THE INDUCTIVE FEATURIZATION WITHHOLDS, NOT MEMORIZED BUSINESS IDENTITY. ROWS ARE ORDERED BY THAT GAP.

Model	Rev.-disj.	Bus.-disj.	Gap
Text-only MLP	0.743	0.712	+0.032
TF-IDF + LogReg	0.762	0.700	+0.062
TF-IDF + LinearSVM	0.740	0.666	+0.074
Relation GNN	0.938	0.816	+0.122
ReviewGuard (Fusion)	0.856	0.730	+0.126
Behavior-only MLP	0.889	0.706	+0.183
Behavior + LogReg	0.782	0.599	+0.183
Behavior + RandomForest	0.934	0.646	+0.289

The pattern is consistent, interpretable, and ordered exactly by how much business identity each model can reach. The text-only MLP loses 0.032 AUC, because review text identifies its business only weakly. The two TF-IDF baselines lose 0.062 and 0.074. ReviewGuard, which sees business aggregates diluted among 266 text dimensions, loses 0.126. The behavior-only models lose 0.183, and the behavioral random forest — the model best equipped to carve up a small set of business-level aggregates — loses 0.289, falling from the best model in the comparison protocol (0.934) to the worst in the primary one (0.646).

That reversal is striking. Under the reviewer-disjoint protocol, the natural reading of Table II is that behavioral features dominate textual ones for fake review detection on YelpCHI, and that a random forest over 19 behavioral features is the model to beat. Under the business-disjoint protocol that conclusion inverts.

The tempting conclusion — and the one an earlier version of this work drew — is that most of what the behavior branch contributes is knowing which businesses attract spam rather than knowing what a spammer does. That reading does not survive the second axis.

B. Business-Disjoint Performance

Table III reports full metrics under the primary protocol in the inductive regime — the strictest cell of the two-by-two, and a lower bound rather than an estimate. Section VI-E gives the transductive counterpart.

ReviewGuard leads on every metric, at 0.730 AUC-ROC and 0.626 Macro-F1. These are modest numbers, and we present them as a floor rather than as a competitive result: detecting a fake review at a business one has never seen, from text and reviewer statistics alone, without usable timestamps, and with the business itself described only by a training-set prior, is a deliberately unforgiving setting. The ordering of the two single branches is worth noting — text (0.712) slightly ahead of behavior (0.706), the reverse of their ordering under the leakier protocol — but it should be read as a statement about this cell of the design rather than about the two modalities, and Section VI-E revisits it under the transductive regime.

C. Does Fusion Help?

Table IV tests ReviewGuard against each baseline.

Fusion improves on every baseline, and the improvement over the classical models is large: $+0.084$ AUC over the behavioral random forest and $+0.131$ over behavioral logistic regression. Against its own ablations the margin is smaller but consistent. ReviewGuard beats the text-only MLP on five of five folds ($+0.018$ mean AUC) and the behavior-only MLP on four of five ($+0.023$).

We are deliberate about what this does and does not establish. With five paired folds the Wilcoxon signed-rank test bottoms out at $p = 0.0625$, so the five-of-five comparisons sit at the floor of what the test can report and nothing here clears $\alpha = 0.05$. The effect is also small relative to fold-to-fold spread, which the standard deviations in Table III show directly.

It also does not survive the second axis, and we report that plainly because it is the more informative outcome. Under the transductive regime the ordering reverses and does so unambiguously: the behavior-only MLP reaches 0.917 against ReviewGuard's 0.853, and beats it on *five of five* folds. Fusion is not merely failing to help there; it is actively costing 0.064 AUC.

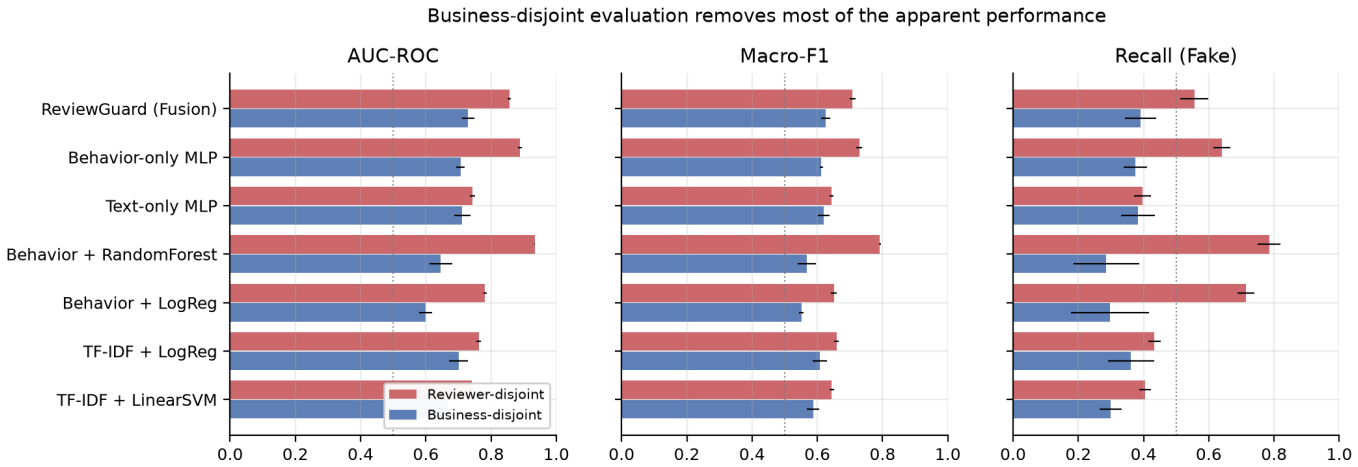

Fig. 2. The same models under both protocols. Bars are means over five folds with standard deviation whiskers. Behavioral models lose most of their apparent performance once businesses no longer span the split; text-based models, which have less access to business identity, lose less.

TABLE III
FIVE-FOLD CROSS-VALIDATED PERFORMANCE ON YELPCHI UNDER THE BUSINESS-DISJOINT PROTOCOL (MEAN ± S.D. ACROSS FOLDS). DECISION THRESHOLDS FOR EVERY MODEL ARE TUNED ON A HELD-OUT VALIDATION SLICE TO MAXIMIZE MACRO-F1.

Model	AUC-ROC	AP	Macro-F1	Rec(Fake)
TF-IDF + LinearSVM	0.666 ± 0.024	0.265 ± 0.029	0.587 ± 0.019	0.299 ± 0.034
TF-IDF + LogReg	0.700 ± 0.029	0.305 ± 0.038	0.608 ± 0.022	0.361 ± 0.071
Behavior + LogReg	0.599 ± 0.020	0.201 ± 0.010	0.552 ± 0.007	0.296 ± 0.120
Behavior + RandomForest	0.646 ± 0.035	0.221 ± 0.029	0.567 ± 0.028	0.286 ± 0.101
Text-only MLP	0.712 ± 0.025	0.325 ± 0.035	0.620 ± 0.018	0.383 ± 0.051
Behavior-only MLP	0.706 ± 0.014	0.295 ± 0.013	0.612 ± 0.005	0.375 ± 0.035
ReviewGuard (Fusion)	**0.730 ± 0.019**	**0.337 ± 0.031**	**0.626 ± 0.015**	**0.391 ± 0.048**

TABLE IV
WILCOXON SIGNED-RANK TESTS OF REVIEWGUARD AGAINST EACH BASELINE OVER THE 5 BUSINESS-DISJOINT FOLDS. Δ IS REVIEWGUARD'S MEAN PER-FOLD ADVANTAGE. AT $n = 5$ THE SMALLEST ATTAINABLE TWO-SIDED p-VALUE IS 0.0625, SO 0.0625 DENOTES A CLEAN SWEEP OF ALL 5 FOLDS.

	AUC-ROC		Macro-F1	
Baseline	**Δ**	p	**Δ**	p
TF-IDF + LinearSVM	+0.064	0.0625	+0.038	0.0625
TF-IDF + LogReg	+0.030	0.0625	+0.018	0.0625
Behavior + LogReg	+0.131	0.0625	+0.074	0.0625
Behavior + RandomForest	+0.084	0.0625	+0.058	0.0625
Text-only MLP	+0.018	0.0625	+0.006	0.1250
Behavior-only MLP	+0.023	0.1250	+0.014	0.1250

The mechanism is visible in the two branches. Under the inductive regime the behavior branch is crippled to 0.706 while text sits at 0.712, so concatenating them combines two comparably weak signals and helps. Under the transductive regime behavior reaches 0.917 while text is unchanged at 0.712, and concatenation now dilutes a strong 19-dimensional signal across 266 largely uninformative text dimensions. Naive concatenation fusion is a reasonable choice when branches are comparable and a poor one when they are not, and which of those regimes one is in was, in our case, determined by a featurization decision rather than by anything about the data.

The honest summary is therefore that we find no regime-independent fusion benefit. Whether a transformer text branch would change this — by raising the text branch to where concatenation is again a fair fight — is exactly the experiment we were unable to run, and it is now the most interesting thing we cannot answer.

D. Discrimination and Errors

Pooling the out-of-fold predictions of all five business-disjoint folds, so that every review is scored once by a model that never saw its business, ReviewGuard recovers 37.3% of fake reviews at the threshold maximizing Macro-F1 on the pooled scores. The error profile is what that objective implies: under 14.5% prevalence, tuning for Macro-F1 buys fake-class recall at a substantial cost in false positives. For deployment the trade-off should be set from the relative cost of suppressing a genuine review against missing a fake one, not from Macro-F1.

E. Feature Regime: Removing Leakage vs. Removing Information

Table II holds the split fixed as the only variable and finds a large gap. But the inductive regime it uses does something stronger than removing leakage: for a business absent from

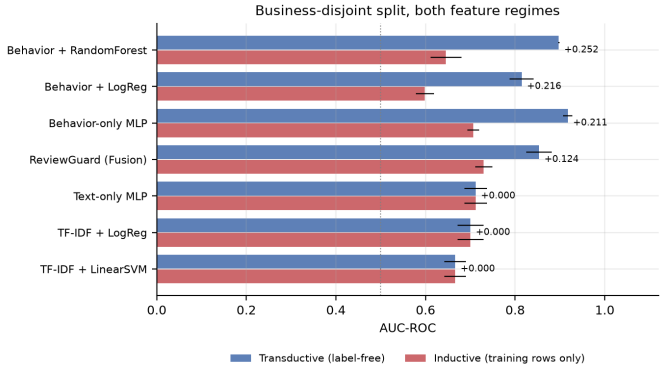

Fig. 3. The business-disjoint split under both feature regimes, ordered by the gain. The three text-based models move by exactly zero — they consume no business aggregates, so the regime cannot reach them — while every behavioral model moves by more than 0.2. One axis, one family.

training, every business-level feature becomes one constant. The model is not merely prevented from recognizing a known spam target — it is prevented from observing the business at all.

Table V repeats the business-disjoint protocol under both feature regimes. The transductive regime changes nothing about the split, the models, or the tuning, and uses no labels; it only permits a held-out business to be described by its own reviews, as a deployed system would observe it.

TABLE V
AUC-ROC UNDER THE BUSINESS-DISJOINT SPLIT IN BOTH FEATURE REGIMES. THE SPLIT, MODELS AND TUNING ARE IDENTICAL; ONLY THE SOURCE OF THE REVIEWER AND BUSINESS AGGREGATES DIFFERS, AND NEITHER REGIME USES A LABEL. THE GAIN IS THE COST OF THE STRICT REGIME – INFORMATION WITHHELD RATHER THAN LEAKAGE REMOVED.

Model	Inductive	Transductive	Gain
TF-IDF + LinearSVM	0.666	0.666	+0.000
TF-IDF + LogReg	0.700	0.700	+0.000
Behavior + LogReg	0.599	0.814	+0.216
Behavior + RandomForest	0.646	0.898	+0.252
Text-only MLP	0.712	0.712	+0.000
Behavior-only MLP	0.706	0.917	+0.211
ReviewGuard (Fusion)	0.730	0.853	+0.124

Two features of the result establish that the axis measures what it claims. First, the three text-based models move by $+0.000$ — not approximately, but to three decimals. The regime governs only reviewer and business aggregates, and text models consume none, so a nonzero movement there would have indicated a bug rather than a finding. Second, every behavioral model moves by a great deal: $+0.211$ for the behavior-only MLP, $+0.216$ for behavioral logistic regression, $+0.252$ for the random forest. One axis moves one family and leaves the other exactly untouched.

The magnitudes force a revision of Table II. Take the random forest, whose collapse was the most dramatic: 0.289 AUC lost when businesses stopped spanning the split. Restoring label-free business profiles returns 0.252 of that, leaving 0.037 — so roughly *87%* of what looked like leakage was our featurization withholding observable information, and about 13% was business identity genuinely unavailable for an unseen business. For the behavior-only MLP the transductive regime more than compensates: it recovers 0.211 against a collapse of 0.183, finishing 0.028 *above* the figure the leaky protocol produced. That is not paradoxical. Under the reviewer-disjoint split a business straddles the fold boundary and its profile is built from whichever of its reviews happened to land in training; under the business-disjoint split with transductive features the profile is complete. A strict split with honest features can beat a loose split with crippled ones.

We consider this the paper's most useful correction, and we arrived at it by making the mistake first. A protocol that removes information along with leakage makes a method look worse than it is, publishes a number that understates it, and gives no signal that anything has gone wrong — the experiment runs, the folds are clean, the result is simply too low. Grouping folds is not sufficient on its own; what a feature is permitted to be computed from has to be stated alongside the split.

F. The Protocol Gap Is Not a Property of a Model

Running the remaining cell of the design makes the point sharper than we expected. Table VI measures the business-disjoint split's cost *separately inside each regime* — that is, it asks how much a model loses to unseen businesses when its features are held fixed.

TABLE VI
THE COST OF THE BUSINESS-DISJOINT SPLIT, MEASURED SEPARATELY INSIDE EACH FEATURE REGIME. TEXT MODELS ARE UNAFFECTED BY THE REGIME AND THEIR GAP IS IDENTICAL IN BOTH COLUMNS. BEHAVIORAL MODELS, WHOSE GAP LOOKS RUINOUS UNDER THE INDUCTIVE REGIME, TRANSFER TO UNSEEN BUSINESSES *better* THAN TEXT MODELS ONCE THEIR FEATURES ARE COMPUTED THE WAY A PLATFORM WOULD COMPUTE THEM.

	Protocol gap (AUC)	
Model	Inductive	Transductive
Behavior + LogReg	0.183	0.005
Behavior-only MLP	0.183	0.013
ReviewGuard (Fusion)	0.126	0.029
Text-only MLP	0.032	0.032
Behavior + RandomForest	0.289	0.045
TF-IDF + LogReg	0.062	0.062
TF-IDF + LinearSVM	0.074	0.074

For text models the two columns are identical, as they must be. For behavioral models they are not remotely alike. The behavior-only MLP loses 0.183 AUC to the split under the inductive regime and 0.013 under the transductive one; the random forest loses 0.289 and 0.045; behavioral logistic regression loses 0.183 and 0.005.

The ordering inverts. Under the inductive regime the behavioral models occupy the three worst positions in the table and the text models the best, which reads as a clean demonstration that behavioral features are the ones dependent on business identity. Under the transductive regime that ordering largely reverses: the two strongest behavioral models take the two

best positions in the table (0.005 and 0.013, against 0.032 for the best text model), and the behavioral family averages 0.021 against 0.056 for text. The reversal is not uniform — the random forest, at 0.045, still trails every text model but the two TF-IDF baselines — but the family-level ordering inverts. Given label-free features, behavioral models generalize to unseen businesses *better* on average than textual ones do, while also scoring far higher in absolute terms (0.917 against 0.712).

So the conclusion our own first pass reached — that behavioral performance on YelpCHI is largely business memorization — is not merely overstated. It is backwards, and the thing that made it look true was a featurization choice rather than anything in the data. We state it this bluntly because the failure is instructive: the protocol gap is not a property of a model or of a feature family. It is a property of the pair (model, feature regime), and measuring it in one regime while reporting it as a property of the model produces a diagnosis that can be wrong in its ordering, not just its magnitude.

G. Graph Models and the Limits of a Business-Disjoint Split

Graph methods sharpen the same point, because for them the distinction between the two feature regimes is not a choice. Message passing over the review graph aggregates over a node's neighbors, and in the R-S-R relation those neighbors are other reviews of the same business. A transductive GNN therefore reconstructs a business profile whether or not its feature matrix was allowed to contain one.

Two measurements make this concrete. First, where a held-out review's same-business neighbors live: under a reviewer-disjoint split 80.1% of them sit in the training set, and under a business-disjoint split 0.0% do. Second, what the resulting structure is worth: under a business-disjoint split, a node's R-S-R degree alone — a count the split was meant to withhold — scores a mean AUC of 0.708, while the same quantity presented as the feature `prod_review_count` under the inductive regime is a constant worth 0.500.

TABLE VII
GRAPH MODELS AND THE CONTROL THAT ISOLATES WHAT THEIR EDGES CARRY. THE CONTROL IS THE BEHAVIOR MLP GIVEN ONLY THE TWO FULL-GRAPH DEGREES. NOTE THE LAST ROW: THE SPLIT CHANGES WHAT A DEGREE IS WORTH BY 0.001, BECAUSE A DEGREE IS READ OFF THE WHOLE GRAPH EITHER WAY. THE SPLIT REMOVES BUSINESS INFORMATION FROM THE FEATURE MATRIX, NOT FROM THE TOPOLOGY.

Model	Rev.-disj.	Bus.-disj.	Gap
Relation GNN	0.938	0.816	+0.122
Behavior MLP	0.889	0.706	+0.183
+ graph degree	0.883	0.788	+0.095
R-S-R degree alone	0.709	0.708	+0.001

Table VII reports the consequence, and its last row is the most direct statement of the problem in this paper. A node's R-S-R degree is worth 0.709 AUC under the reviewer-disjoint split and 0.708 under the business-disjoint one — the split moves it by 0.001. Every other quantity in this study moves substantially between protocols, because every other quantity is computed from rows the split can partition. A degree is read off the whole graph either way.

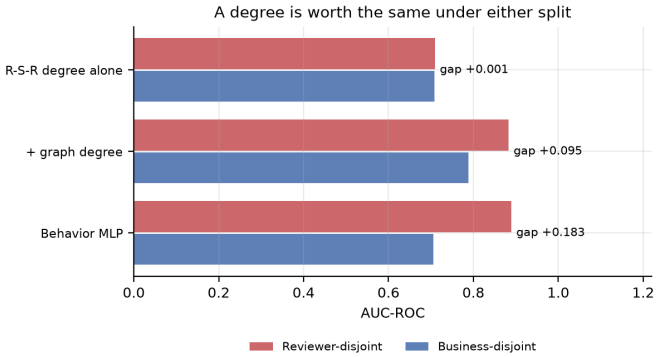

Fig. 4. What the split does and does not reach. Behavioral features lose 0.183 AUC when businesses stop spanning the split; the same features plus two graph degrees lose 0.095; a graph degree on its own loses 0.001. The split partitions rows, and a degree is not computed from rows it can partition.

The rows above show what follows. Under the reviewer-disjoint split the behavior MLP already scores 0.889 and adding the degrees changes nothing (-0.006): its feature matrix, fitted on training rows that include the very businesses being scored, already carries what they would supply. Under the business-disjoint split the same features fall to 0.706 and the same two degrees add $+0.082$. The split removed business information from the feature matrix and left it in the topology. Ordering the gap column ranks the four rows by exactly how much of each model's information comes from partitionable rows: $+0.183$ for the feature-only MLP, $+0.095$ and $+0.122$ for the graph-informed models, $+0.001$ for a degree.

Two consequences. First, the GNN reaches 0.816 where its features alone reach 0.706, and the degree control reaches 0.788 — three quarters of the distance — so the larger part of its advantage is a count a feature model can be handed directly, with the remainder the honest size of the relational contribution. Second, and with the sign opposite to intuition, the GNN is *less* disturbed by the stricter protocol than the plain feature model, losing 0.122 against 0.183. That is not better generalization to unseen businesses; it is a larger share of its information living where the split cannot reach. A model can be made to look robust to a leakage control by drawing what it needs through a channel the control does not govern.

The implication is narrower than "graph methods leak" and more awkward: a business-disjoint split is necessary but not sufficient for a transductive graph model, because the topology re-encodes what the split removed from the features. Reporting a business-disjoint number for such a model, without stating what its edges carry, does not settle the question the split was posed to answer.

H. What the Behavior Branch Actually Uses

SHAP [16] attribution over the behavior-only MLP agrees with the degree control from the feature side. One input dominates: `prod_review_count`, the number of reviews the business has received, carries a mean absolute

SHAP value of 0.462, five times that of the next feature (`review_word_count`, 0.093), and the four `prod_*` features account for 55.5% of total attribution while making up 4 of 19 inputs. The most influential input to the behavior branch is a property of the business, not of the review or its author — which is why the choice of what a business-level feature may be computed from moves this branch so much and text models not at all.

VII. Discussion

A. Implications for Benchmarking

We do not claim that prior YelpCHI results are wrong. We claim that the standard protocol does not separate three capabilities that behave very differently in deployment: recognizing deception, recognizing that a *particular* business has been targeted before, and recognizing the kind of business that attracts targeting. The first and third transfer to a new market; the second does not. A detector evaluated only under a business-overlapping split has not been shown to do the first or third, and its reported number will not survive contact with a new market.

The obvious correction — group folds by business — is right but incomplete, and the way it is incomplete is the more useful half of our result. Grouping alone leaves two loose ends. If features are computed strictly from training rows, the split removes the third capability along with the second, and the resulting number understates the method rather than correcting it. And if the model consumes a graph, grouping does not remove the second capability at all, because the topology reconstructs a business profile from same-business neighbors whatever the feature matrix contains. Reporting a business-disjoint number is therefore not on its own evidence of anything; what a model's features and edges are permitted to see has to be stated alongside it. Concretely, we suggest reporting the split, the source of aggregate statistics, and — for graph models — a degree control, which costs one extra run.

The point generalizes past this dataset. Any fraud benchmark assembled by sampling entities and collecting their records will concentrate labels by entity. Any model given entity-level aggregates can exploit that, and any model given entity-level *edges* can reconstruct the aggregates.

B. Limitations

Our study has seven limitations we consider material.

The transductive regime is optimistic in a specific way. It describes a held-out business using *all* of its reviews — a median of ten here, and a mean of 38. A listing created yesterday has one or two, so its profile would be far noisier than anything we measure, and the 0.805 transfer figure should be read as an upper bound on what a business profile is worth at the moment a business is new. The honest operating point for a genuinely cold business lies somewhere between our two regimes, and locating it would require a temporal record we do not trust in this distribution.

Our graph result rests on a re-implementation. The model in Section VI-G follows the CARE-GNN family — per-relation mean aggregation over R-U-R and R-S-R — but is not CARE-GNN, and omits its learned neighbor filtering. We report what our own GNN does; whether published graph detectors behave the same way under a business-disjoint split is a question our degree control motivates rather than answers, and running it on released implementations is the single most valuable follow-up we can name.

The text branch is lexical rather than transformer-based, for the infrastructural reason given in Section IV. Our absolute numbers on the text side should be read as a floor.

Timestamps are excluded because they are contaminated in the distribution available to us, which removes the burstiness features that prior work found valuable. A study using a clean temporal record would likely obtain a stronger behavior branch, and should re-examine the protocol gap with those features included — burstiness is plausibly less business-specific than the aggregates we use, and may transfer better.

Five folds admit no significance at $\alpha = 0.05$ under a signed-rank test, whose smallest attainable two-sided p-value at $n = 5$ is 0.0625. Our conclusions rest on effect sizes and fold-level consistency, and the fusion comparison in particular is underpowered: a study wanting to settle it should run repeated cross-validation or more folds.

All results are on a single corpus. YelpCHI has 1,224 businesses, so the business-disjoint folds hold roughly 244 businesses out each — enough to measure a gap, but a single market and a single collection methodology. Whether the three-way separation holds on YelpNYC or YelpZIP is untested here.

Finally, YelpCHI's labels originate in Yelp's proprietary filter. They encode one platform's operational definition of spam, with unknown false negative rate, and both our results and the leakage we document are properties of that labeling as much as of deception itself.

C. Ethical Considerations

At the operating points reported here, false positives substantially outnumber true positives, and a false positive suppresses a genuine customer's review. A system of this accuracy is appropriate for ranking a human moderation queue, not for automated removal. We report Macro-F1 and per-class metrics rather than accuracy specifically so that minority-class behavior stays visible.

VIII. Conclusion

We set out to build a text-plus-behavior fake review detector for YelpCHI and found that the benchmark's label structure makes conventional evaluation misleading — and then that the obvious correction is misleading in its own way.

Fake reviews cluster by business to the point that the per-business fake rate is a 0.951-AUC predictor, so grouping folds by business is necessary. It is not sufficient, in two directions. On the feature side, business-level quantities are of three kinds: label-derived statistics are leakage, the identity of a known

target does not transfer, and a label-free profile of a business's own reviews transfers to unseen businesses at 0.805 AUC. A strict training-only featurization removes all three, and doing so is what made our own first pass conclude that behavioral features were mostly memorization. Measured with those profiles restored, the split costs behavioral models 0.021 AUC on average against 0.056 for text — reversing the ordering at the family level. On the model side, a transductive graph model's edges reconstruct the profile whatever its features hold: a degree is worth 0.709 AUC under one split and 0.708 under the other, and a plain MLP given two degrees recovers three quarters of a GNN's advantage.

The general form is that a protocol gap belongs to the pair (model, feature regime), not to a model. Reported as a model property it can invert an ordering rather than merely rescale it, and nothing in the experiment signals that this has happened. We recommend reporting the split, the source of aggregate statistics, and — for graph models — a degree control.

ReviewGuard itself is a negative result: it leads its ablations under one regime and loses to its behavior branch on five of five folds under the other, so we find no regime-independent fusion benefit. We additionally document label-contaminated timestamps and an uninformative behavioral feature file in a widely circulated distribution of the dataset.

Three directions follow. Completing the transformer text branch, which our infrastructure prevented, and which would test whether a stronger text branch restores fusion. Re-examining published YelpCHI and YelpNYC results along both axes, graph methods especially. And obtaining a clean temporal record, so that burstiness can be assigned to one of the three categories — unlike the static aggregates here it is plausibly a property of a campaign rather than a business, and would then transfer where they do not.

Acknowledgment

The authors thank Prof. Song Fang of the School of Computer Science, University of Oklahoma, for guidance throughout this project.

Reproducibility

Code, the data integrity audit, and the metrics behind every table and figure in this paper are available at

https://github.com/RohanMukka/Combining-Transformer-Semantics-and-Reviewer-Behavior-for-Fake-Review-Detection-on-Yelp

All result tables are generated programmatically from the recorded metrics files rather than transcribed by hand, and both evaluation protocols run from a single command each.

References

[1] M. Luca and G. Zervas, "Fake it till you make it: Reputation, competition, and Yelp review fraud," *Management Science*, vol. 62, no. 12, pp. 3412–3427, 2016.

[2] E. Crothers, N. Japkowicz, and H. Viktor, "Machine-generated text: A comprehensive survey of threat models and detection methods," *IEEE Access*, vol. 11, pp. 70977–71002, 2023.

[3] S. Rayana and L. Akoglu, "Collective opinion spam detection: Bridging review networks and metadata," in *Proc. ACM KDD*, Sydney, Australia, 2015, pp. 985–994.

[4] M. Ott, Y. Choi, C. Cardie, and J. T. Hancock, "Finding deceptive opinion spam by any stretch of the imagination," in *Proc. ACL*, Portland, OR, 2011, pp. 309–319.

[5] N. Jindal and B. Liu, "Opinion spam and analysis," in *Proc. ACM WSDM*, Palo Alto, CA, 2008, pp. 219–230.

[6] A. Mukherjee, V. Venkataraman, B. Liu, and N. S. Glance, "What Yelp fake review filter might be doing?," in *Proc. ICWSM*, Cambridge, MA, 2013, pp. 409–418.

[7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, Minneapolis, MN, 2019, pp. 4171–4186.

[8] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," arXiv:1907.11692, 2019.

[9] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. ICLR*, Toulon, France, 2017.

[10] Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng, and P. S. Yu, "Enhancing graph neural network-based fraud detectors against camouflaged fraudsters," in *Proc. ACM CIKM*, Virtual Event, Ireland, 2020, pp. 315–324.

[11] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.

[12] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, "Shortcut learning in deep neural networks," *Nature Machine Intelligence*, vol. 2, no. 11, pp. 665–673, 2020.

[13] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE ICCV*, Venice, Italy, 2017, pp. 2980–2988.

[14] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, San Diego, CA, 2015.

[15] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[16] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. NeurIPS*, Long Beach, CA, 2017, pp. 4765–4774.

[17] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.